Comprehensive Report: CNF Representation and Resolution-Based Inference in a Wumpus World Agent

1. Introduction

This report explains the internal working of a Knowledge-Based Agent designed for the Wumpus World environment. The agent uses **Propositional Logic**, **Conjunctive Normal Form (CNF)**, and a **Resolution-based inference mechanism** to reason about safe and unsafe cells in a grid.

The goal of the agent is to safely navigate the environment by logically deducing:

- Which cells contain pits or the Wumpus
- Which cells are safe to move into

---

2. Knowledge Base (KB) and CNF Structure

The core of the agent's reasoning system is its **Knowledge Base (KB)**. All knowledge is stored in **Conjunctive Normal Form (CNF)**.

CNF Representation

In CNF:

- The KB is a conjunction (**AND**) of clauses
- Each clause is a disjunction (**OR**) of literals

Implementation Representation

- **Literal** → String
  - `"P_1_2"` → Pit at (1,2)
  - `"~P_1_2"` → No Pit at (1,2)
- **Clause** → Python List
  - Example:
    `["~B_1_1", "P_1_2", "P_2_1"]`
    Represents:
    $\neg B_{11} \lor P_{12} \lor P_{21}$
- **Knowledge Base (KB)** → List of Clauses
  - Each clause is implicitly ANDed with others

---

3. The TELL Operation: Translating Percepts into CNF

When the agent enters a cell, it receives percepts:

- **Breeze (B)** → Indicates nearby pit
- **Stench (S)** → Indicates nearby Wumpus

These percepts are converted into logical rules and stored in the KB.

3.1 Breeze Rule Transformation

Logical Rule

A cell is breezy **iff** at least one adjacent cell contains a pit:

[
B_{r,c} \Leftrightarrow (P_{n1} \lor P_{n2} \lor \dots)
]

Step 1: Remove Biconditional

Split into two implications:

Part A

[
B_{r,c} \Rightarrow (P_{n1} \lor P_{n2} \lor \dots)
]

CNF form:
[
\neg B_{r,c} \lor P_{n1} \lor P_{n2} \lor \dots
]

Code Representation:

```
world.KB.append([f"~B_{r}_{c}"] + p_lits)
```

Part B

[
(P_{n1} \lor P_{n2} \lor \dots) \Rightarrow B_{r,c}
]

Convert using De Morgan's Law:

[
(\neg P_{n1} \lor B_{r,c}) \land (\neg P_{n2} \lor B_{r,c}) \land \dots
]

Code Representation:

```
for pl in p_lits:
    world.KB.append([f"B_{r}_{c}", f"~{pl}"])
```

3.2 Stench Rule

The exact same transformation applies to the Wumpus:

[
S_{r,c} \Leftrightarrow (W_{n1} \lor W_{n2} \lor \dots)
]

3.3 Percept Assertion

The actual percept is also added as a **unit clause**:

- Breeze present → `["B_1_1"]`
- No Breeze → `["~B_1_1"]`

This grounds the logical rules in reality.

---

4. The ASK Operation: Proof by Contradiction

Before moving into a new cell, the agent must determine if it is safe.

Strategy: Proof by Contradiction

To prove a statement, the agent:

1. Assumes the opposite is true
2. Adds that assumption to the KB
3. Runs resolution

Example

To prove:
[
\neg P_{2,2}
]

The agent assumes:
[
P_{2,2}
]

- If this leads to a contradiction → assumption is false → cell is safe
- Otherwise → cannot prove safety

---

5. Resolution Algorithm (resolution_refutation)

This is the core reasoning engine of the agent.

5.1 Resolve Function

Resolution works by eliminating complementary literals.

Rule

[
(A \lor B), (\neg A \lor C) \Rightarrow (B \lor C)
]

Implementation Behavior

- Takes two clauses
- Finds complementary literals (e.g., `A` and `~A`)
- Removes them
- Merges remaining literals

Example

```
["A", "B"] and ["~A", "C"] → ["B", "C"]
```

5.2 Set of Support Strategy (Optimization)

Instead of resolving all clauses with each other (which is inefficient), the algorithm uses a **Set of Support** strategy.

Steps

1. Initialize queue with **query clause**
2. Pop clause `c1` from queue
3. Resolve `c1` with all clauses in KB
4. Add new resolvents to:
    - KB
    - Queue

Why This Matters

- Prevents exponential explosion
- Focuses reasoning only on relevant facts

---

6. Loop Termination Conditions

The resolution loop stops in three cases:

---

1. Contradiction Found (Success)

- Empty clause `[]` is generated
- Indicates logical inconsistency
- Query is **proved TRUE**

---

2. Queue Exhausted (Failure)

- No more clauses to resolve
- Query **cannot be proven**

---

3. Timeout Condition

- Maximum iterations (e.g., 2000) reached
- Prevents infinite loops
- Returns **False safely**

---

7. Decision-Making Strategy

After applying resolution, the agent classifies each cell:

Case 1: Safe Cell

If both are proven:

- ¬P(r,c)
- ¬W(r,c)

➡ Cell is **SAFE**

Case 2: Dangerous Cell

If either is proven:

- P(r,c) OR W(r,c)

➡ Cell is **DANGEROUS**

Case 3: Unknown Cell

If neither can be proven:

➡ Cell remains **UNKNOWN**

---

8. Conclusion

This agent demonstrates how **logical reasoning** can be applied to decision-making in uncertain environments.

Key strengths:

- Uses **CNF for efficient reasoning**
- Applies **Resolution for sound inference**
- Implements **Set of Support for optimization**
- Handles uncertainty via **proof-based classification**

This approach ensures that the agent:

- Avoids unsafe cells whenever possible
- Makes informed decisions based on available knowledge
- Remains computationally efficient